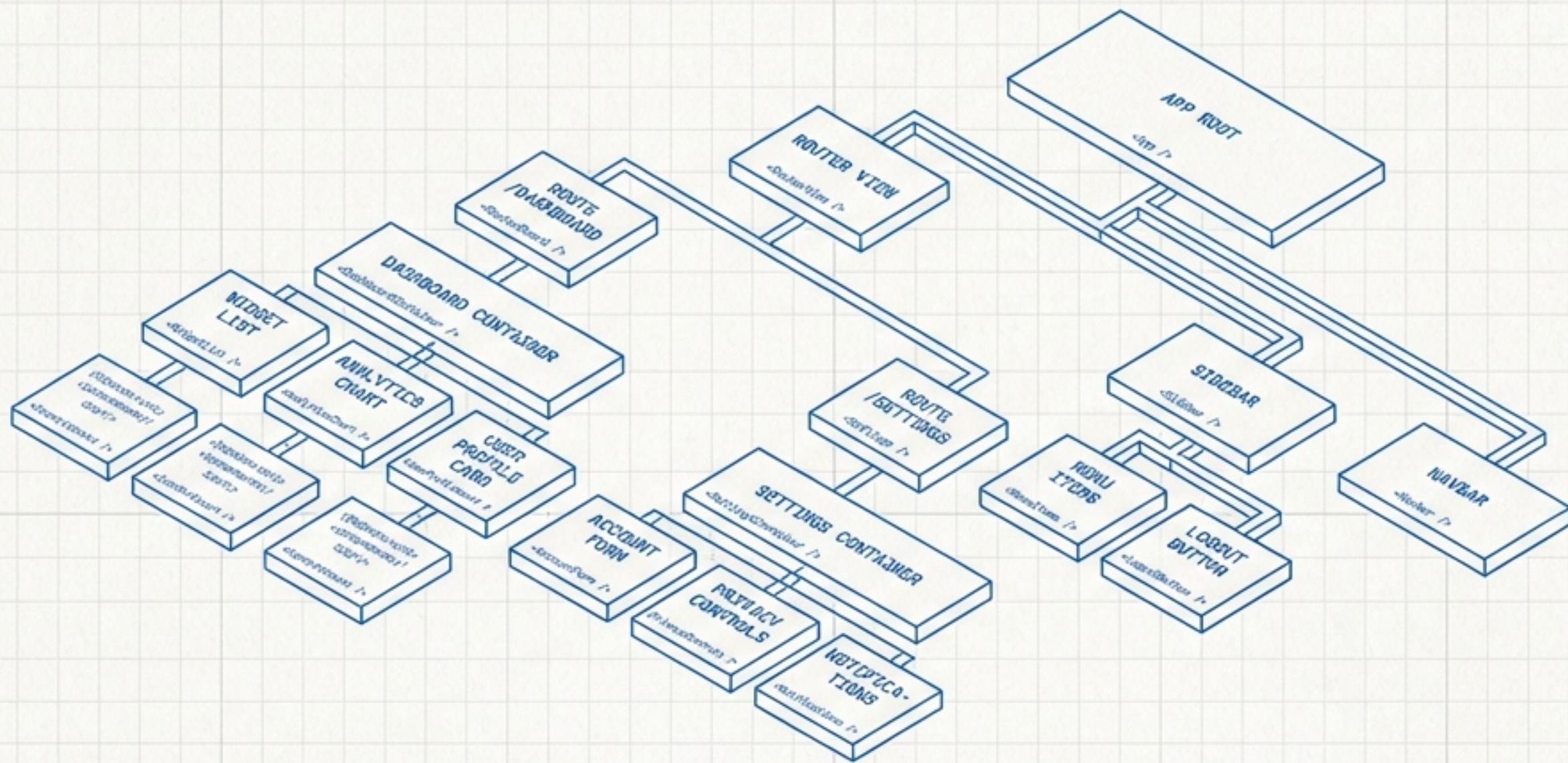
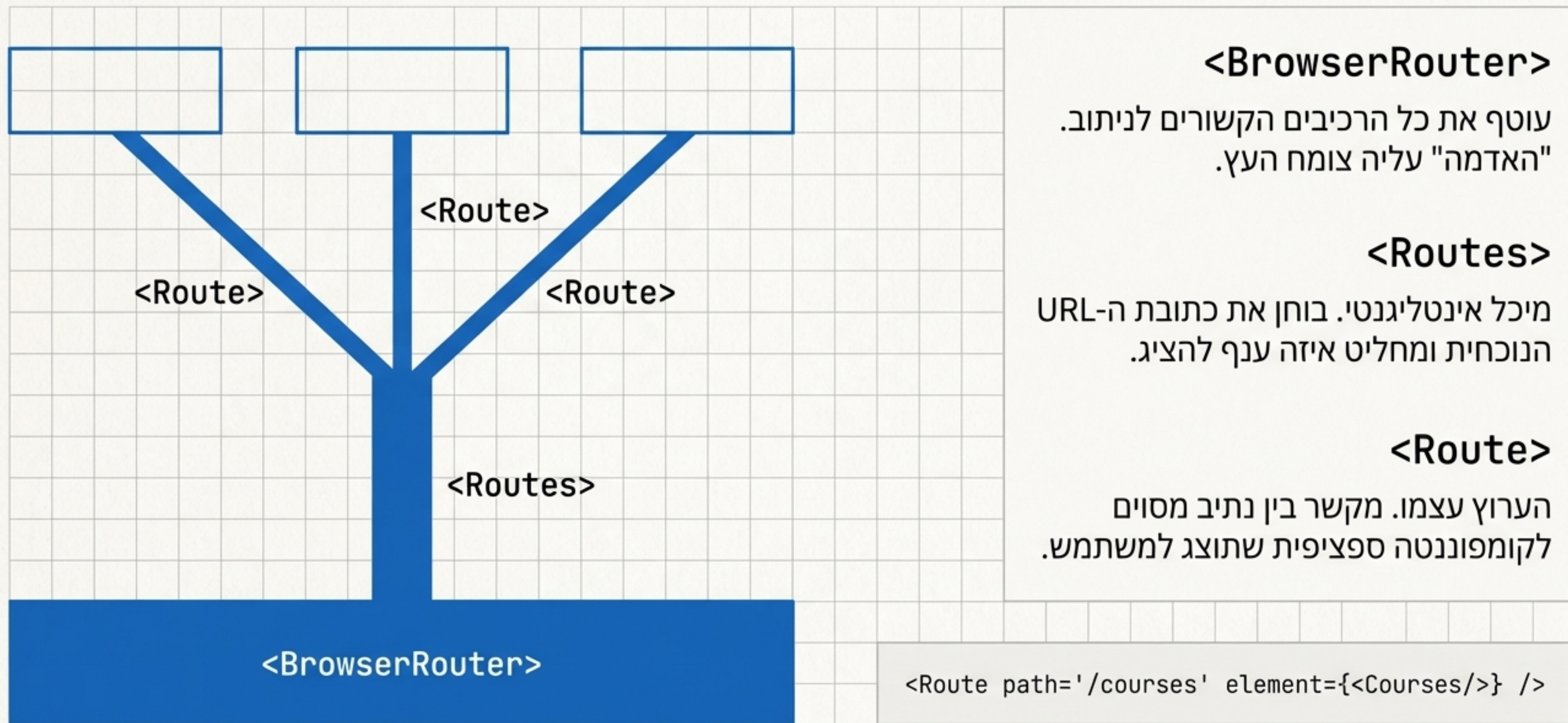




ארגז הכלים של ארכיטקט React: ניתוב, מצב ו-Hooks

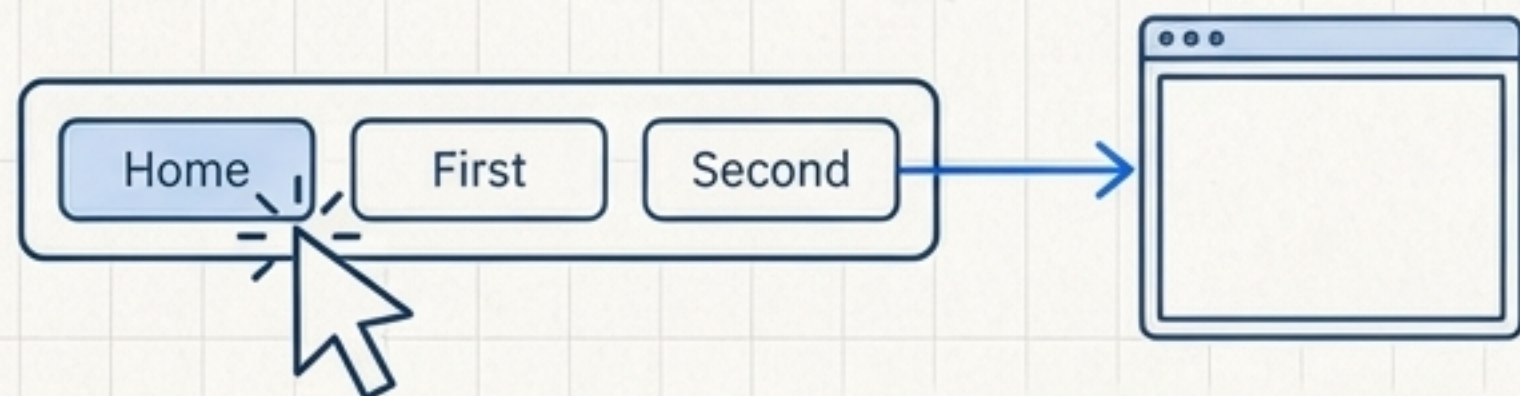
ממיפוי קונספטואלי ועד לביצוע: המדריך השלם לבניית אפליקציות חכמות ויעילות.

שכבת הניווט: אנטומיה של ה-Router



מעבר בין תצוגות: <Link> מול useNavigate

הצהרתי / סטטי



כלי: הקומפוננטה <Link>

טריגר: לחיצה ישירה של המשתמש.

מתי נשתמש? תפריטי ניווט (Navbar), קישורים קבועים, מעבר פשוט מעמוד לעמוד.

```
import React from 'react';
import { Link } from 'react-router-dom';

export default function Nenu() {
  return (
    <div className='row' style={{border:'2px solid gray'}}>
      <div className='col-4'>
        <Link to='/'>Home</Link>
      </div>
      <div className='col-4'>
        <Link to='/First'>First</Link>
      </div>
      <div className='col-4'>
        <Link to='/Second'>Second</Link>
      </div>
    </div>
  );
}
```

אימפרטיבי / דינמי



כלי: ההוק useNavigate()

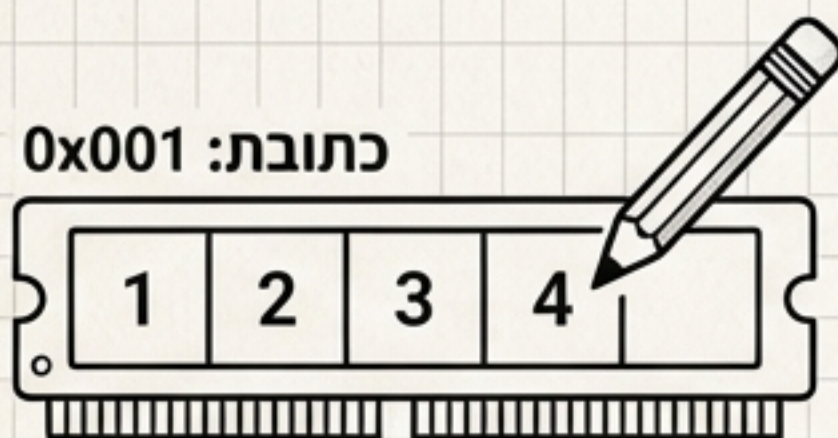
טריגר: לוגיקה של קוד (תנאים, סיום תהליך).

מתי נשתמש? לאחר התחברות מוצלחת (Login), שליחת טופס, או ניתוב המותנה בהרשאות (if/else).

```
const checkPass = () => {
  const user = users.find(user => user.name === name && user.password === pass);
  if(user) {
    setIsLoggedIn(true);
    setLoggedInUserId(user.id);
    nav('/homepage');
  } else {
    alert('Wrong username or password');
  }
};
```

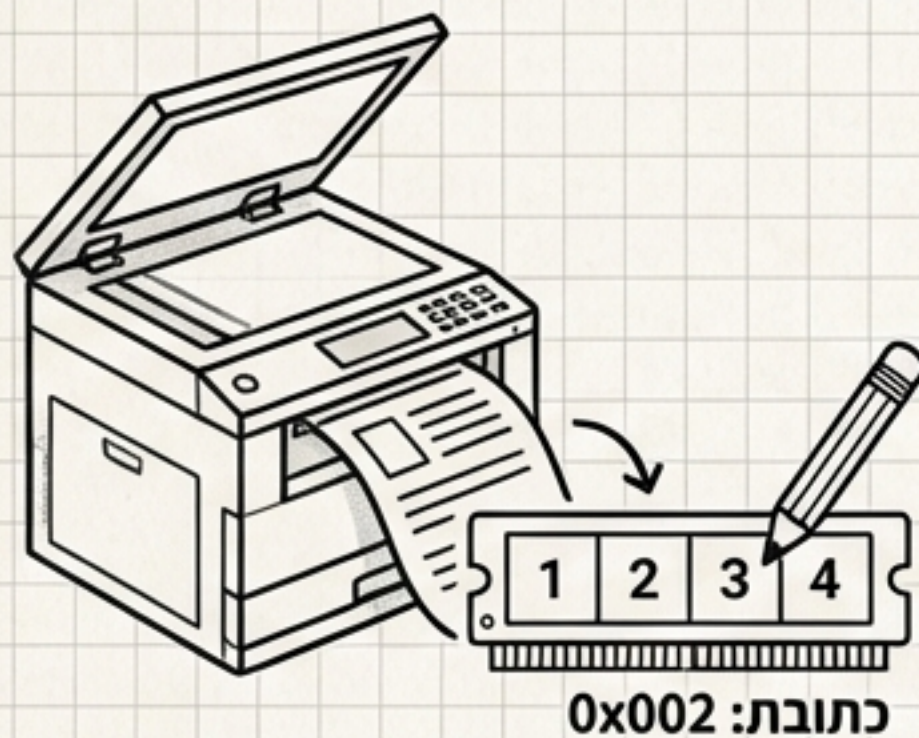

מנוע הליבה: State ועקרון ה-Immutability

למה useState דורש העתקה ולא דריסה? ריאקט לא בודקת את תוכן המערך, אלא את כתובת הזיכרון שלו.



```
const array1 = [1, 2, 3]
const array2 = array1
array2.push(4)
//array1 == [1, 2, 3, 4] וגם array2 == [1, 2, 3, 4]
```

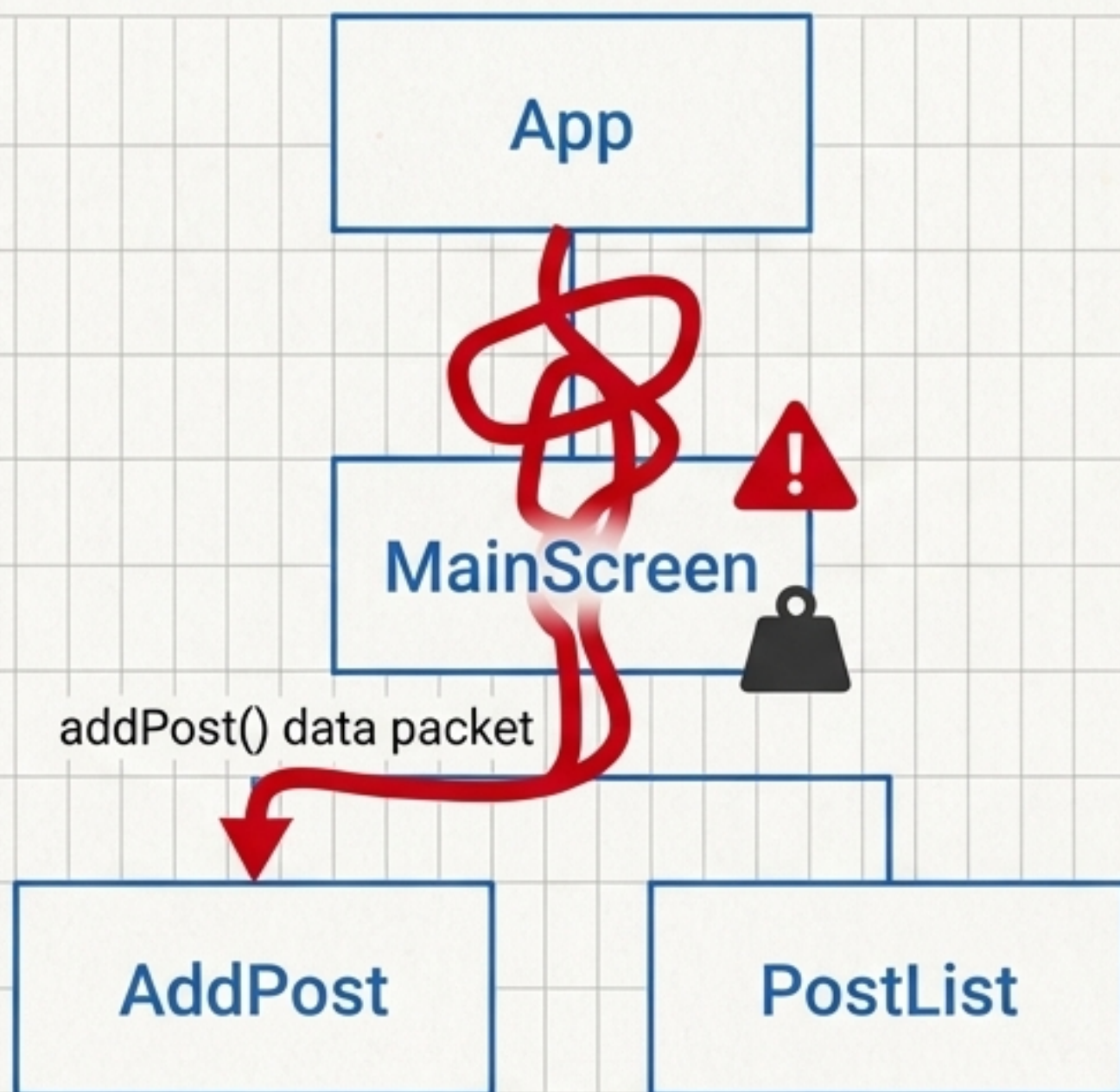
array2.push(4) - הכתובת נשארה זהה, ריאקט מתעלמת.



```
const array1 = [1, 2, 3]
const array2 = [...array1] // העתקה רדודה
array2.push(4)
//array1 == [1, 2, 3] array2 == [1, 2, 3, 4]
```

const array2 = [...array1, 4] העתקה רדודה. נוצרה כתובת זיכרון חדשה.

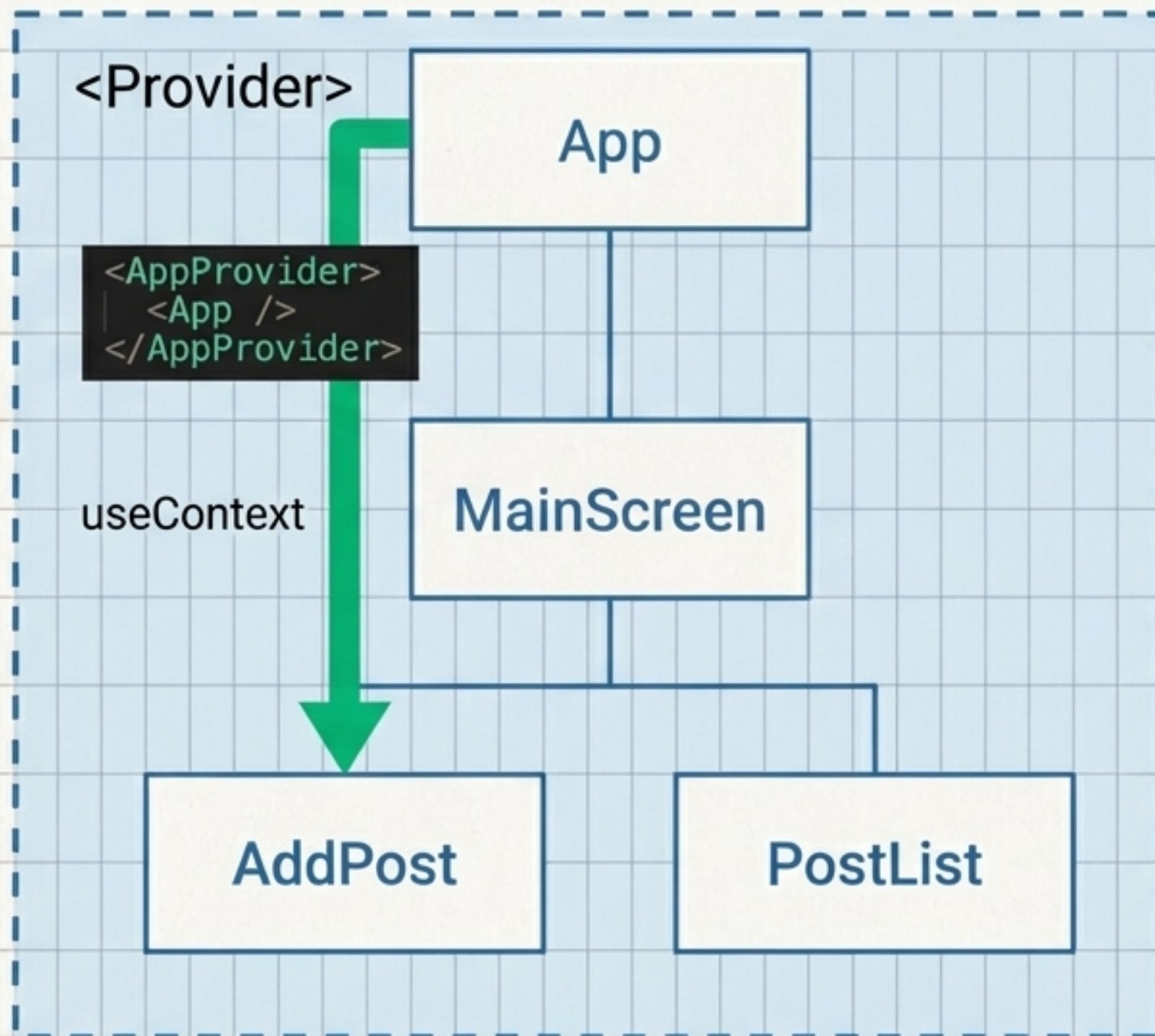
האתגר הארכיטקטוני: הבעיה של Prop Drilling



הבעיה: כאשר מעבירים props לעומק העץ, המידע נאלץ לעבור דרך קומפוננטות ביניים שלעיתים כלל לא משתמשות בו (כמו MainScreen).

התוצאה: קוד מסורבל, קשה לתחזוקה, ורכיבים שמכירים מידע שלא רלוונטי אליהם.

הפתרון: שידור ישיר עם Context API



```
import React, { useContext } from 'react'
import { AppContext } from '../contextApi';

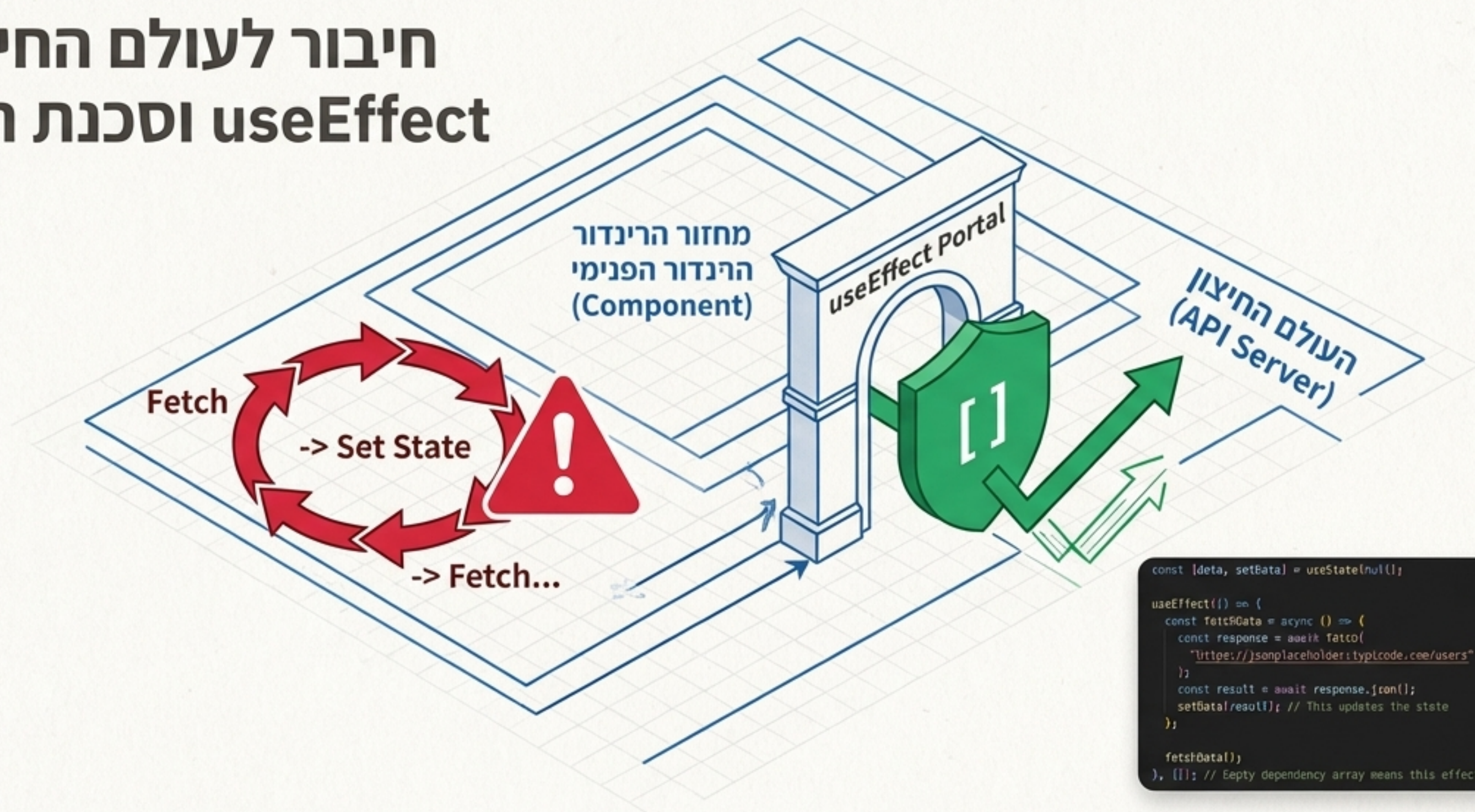
export default function AddPost() {
  const {addPost} = useContext(AppContext);
  //addPost received in child component

  return (
    <div>
      <button>I can add post now!</button>
    </div>
  )
}
```

הפתרון: יצירת אזור גלובלי (createContext) שעוטף את האפליקציה ב- <Provider>.

הקסם: רכיבים שצריכים את המידע פשוט שואבים אותו ישירות באמצעות ההוק useContext, ללא קשר למיקומם בעץ. השרשרת נשברת.

חיבור לעולם החיצון: useEffect וסכנת הלולאות



```
const [data, setData] = useState(null);

useEffect(() => {
  const fetchData = async () => {
    const response = await fetch(
      "https://jsonplaceholder.typicode.com/users"
    );
    const result = await response.json();
    setData(result); // This updates the state
  };

  fetchData();
}, []); // Empty dependency array means this effect runs only once after the initial
```

הרעיון:

פעולות צדדיות (Side Effects) כמו קריאות שרת (fetch) מתבצעות ב-`useEffect` כדי לא להפריע לרנדור של ה-UI.

המלכודת:

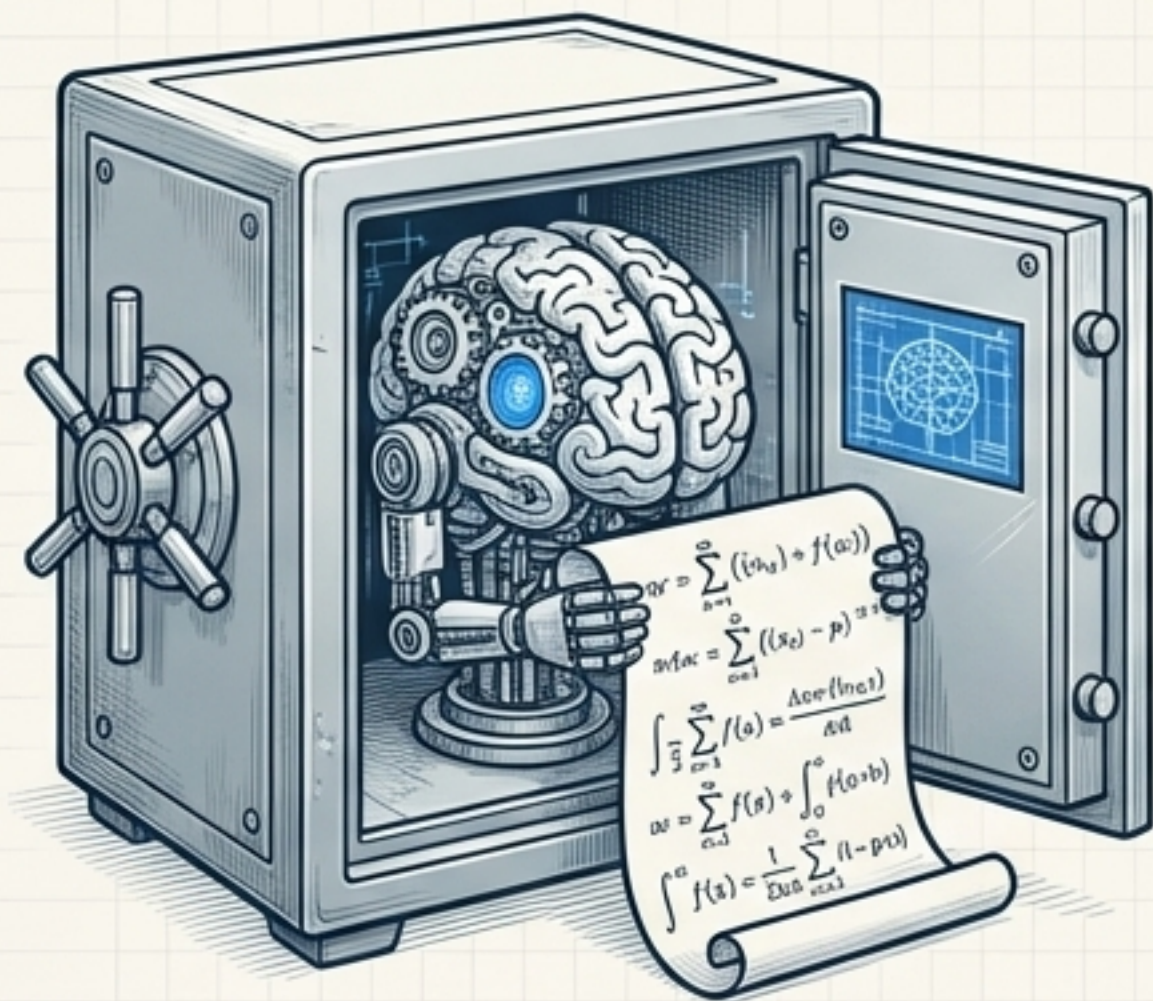
עדכון State בתוך הקומפוננטה גורם לרנדור מחדש. אם ה-`fetch` לא עטוף נכון, ניצור לולאה אינסופית.

המגן:

הוספת מערך תלויות ריק `[]` מבטיחה שהפונקציה תרוץ אך ורק פעם אחת כשהרכיב נטען.

ארגון וייעול מתקדם: useRef-useMemo

useMemo - זיכרון מטמון



המטרה: שמירה בזיכרון של תוצאות חישובים "יקרים" או מורכבים.

היתרון: מונע חישוב חוזר ומיותר בכל רינדור, כל עוד הנתונים לא השתנו. שיפור ביצועים דרמטי.

useRef - עוגן ומצביע



המטרה 1 (עוגן): שמירת נתונים שניתנים לשינוי (Mutable) מבלי לגרום לרינדור מחדש של הקומפוננטה.

המטרה 2 (מצביע): גישה ישירה לאלמנטים ספציפיים ב-DOM (כמו מיקוד על אינפוט).

ה-Sheat Sheet האולטימטיבי: מתי להשתמש באיזה Hook?

Hook	מטרה עיקרית	טריגר לרינדור?	המודל המנטלי (Icon)
useState	שמירת נתונים מעודכנים שמשפיעים על ה-UI	כן (מרנדר מחדש)	מנוע עדכון מסך 
useEffect	ביצוע פעולות צדדיות (API, טיימרים)	לא (רץ אחרי הרינדור)	שער לעולם החיצון 
useMemo	זכירת תוצאות של חישובים כבדים	לא	מוח / מטמון 
useRef	אחיזת נתונים שקטים או גישה ישירה ל-DOM	לא	עוגן חסין רינדור 

הארכיטקטורה המלאה: הכל מתחבר יחד

